

**Федеральное государственное автономное образовательное учреждение
высшего образования**

"Национальный исследовательский университет

"Высшая школа экономики"

Московский институт электроники и математики им. А. Н. Тихонова

Департамент компьютерной инженерии

Отчёт по дисциплине

«Распределённые базы данных и сетевые вычисления»

**Пакет java.net. Средства Java для создания сетевых
(распределённых) приложений**

Студент:

Абрамов Илья Александрович

Группа: МКС254

Преподаватель: Байбикова Т.Н.

1 Задание

Рассмотреть средства Java для работы с сетью. Создайте приложение, позволяющее вывести: protocol; file name; host; path; port. Подготовьте отчет, включите в него свои примеры (листнинги) с кратким описанием.

Создайте многопоточное приложение. В одном потоке обеспечьте считывание содержимого (страницы) с одного web-сайта. Во втором потоке - считывание содержимого (страницы) с другого web-сайта. Подготовьте отчет, включите в него свои примеры (листнинги) с кратким описанием.

2 Разработанная программа

Всего было создано 4 файла:

- URLParser.java - парсер URL
- WebsiteReader.java - читатель веб-сайтов
- WebContentFetcher.java - многопоточный загрузчик
- App.java - класс для демонстрации

3 Листинг

3.1 Файл URLParser.java

```
import java.net.MalformedURLException;
import java.net.URL;

// Task 1: Парсинг компонентов URL
public class URLParser {

    public static void parseURL(String urlString) {
        System.out.println("\n" + "=".repeat(60));
        System.out.println("Parsing URL: " + urlString);
        System.out.println("=".repeat(60));

        try {
            URL url = new URL(urlString);
            displayURLComponents(url);
        } catch (MalformedURLException e) {
            System.err.println("ERROR: Invalid URL format!");
            System.err.println("Reason: " + e.getMessage());
        }
    }

    public static void displayURLComponents(URL url) {
        System.out.println("\n📄 URL Components:");
        System.out.println("-".repeat(60));

        String protocol = url.getProtocol();
        System.out.printf("%-20s: %s\n", "Protocol", protocol);

        String host = url.getHost();
        System.out.printf("%-20s: %s\n", "Host", host);

        int port = url.getPort();
        int defaultPort = url.getDefaultPort();
        if (port == -1) {
            System.out.printf("%-20s: %d (default for %s)\n", "Port", defaultPort, protocol);
        } else {
            System.out.printf("%-20s: %d (explicit)\n", "Port", port);
            System.out.printf("%-20s: %d\n", "Default Port", defaultPort);
        }

        String path = url.getPath();
        System.out.printf("%-20s: %s\n", "Path", path.isEmpty() ? "/" : path);

        String file = url.getFile();
        System.out.printf("%-20s: %s\n", "File", file.isEmpty() ? "/" : file);

        String query = url.getQuery();
        if (query != null && !query.isEmpty()) {
            System.out.printf("%-20s: %s\n", "Query Parameters", query);
        }

        String ref = url.getRef();
        if (ref != null && !ref.isEmpty()) {
            System.out.printf("%-20s: %s\n", "Fragment/Anchor", ref);
        }

        String authority = url.getAuthority();
        System.out.printf("%-20s: %s\n", "Authority", authority);

        System.out.println("-".repeat(60));
        System.out.printf("%-20s: %s\n", "Complete URL", url.toString());
    }

    public static void main(String[] args) {
        System.out.println("\n");
        System.out.println("Task 1: URL Component Parser Demo");
        System.out.println("Java Networking (java.net.URL)");
        System.out.println("\n");

        parseURL("http://www.example.com");
        parseURL("https://www.example.com:8080/api/users");
        parseURL("http://www.example.com/search?q=java&lang=en");
        parseURL("ftp://ftp.example.com/pub/docs/manual.pdf");
        parseURL("http://localhost:3000/index.html");
    }
}
```

```

        parseURL("https://docs.oracle.com/javase/tutorial/networking/urls/index.html#DOWNLOADING");
        parseURL("https://user:pass@www.example.com:8443/path/to/resource.html?id=123&type=test#section2")
    }
}

// Демонстрация обработки ошибок
parseURL("http://invalid url with spaces");

System.out.println("\n" + "=".repeat(60));
System.out.println("URL parsing demonstration completed!");
System.out.println("=".repeat(60) + "\n");
}
}

```

3.2 Файл WebsiteReader.java

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.URL;
import java.net.UnknownHostException;

// Runnable для чтения содержимого веб-сайта в отдельном потоке
public class WebsiteReader implements Runnable {
    private final String urlString;
    private final String threadName;
    private int linesRead = 0;
    private int charsRead = 0;
    private long executionTime = 0;
    private boolean success = false;

    public WebsiteReader(String urlString, String threadName) {
        this.urlString = urlString;
        this.threadName = threadName;
    }

    @Override
    public void run() {
        long startTime = System.currentTimeMillis();

        synchronized (System.out) {
            System.out.println "[" + threadName + "] Starting: " + urlString);
        }

        try {
            URL url = new URL(urlString);

            // Открываем поток для чтения
            try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(url.openStream()))) {

                synchronized (System.out) {
                    System.out.println "[" + threadName + "] Connected successfully");
                }

                String line;
                while ((line = reader.readLine()) != null) {
                    linesRead++;
                    charsRead += line.length();

                    // Имитация обработки данных
                    Thread.sleep(10);
                }

                success = true;
            }
        } catch (UnknownHostException e) {
            synchronized (System.err) {
                System.err.println "[" + threadName + "] ERROR: Unknown host - " + e.getMessage());
            }
        } catch (IOException e) {
            synchronized (System.err) {
                System.err.println "[" + threadName + "] ERROR: I/O error - " + e.getMessage());
            }
        } catch (InterruptedException e) {
            synchronized (System.err) {
                System.err.println "[" + threadName + "] ERROR: Thread interrupted");
            }
        }
    }
}

```

```

        executionTime = System.currentTimeMillis() - startTime;

        synchronized (System.out) {
            if (success) {
                System.out.println "[" + threadName + "] Completed: " + linesRead +
                    " lines, " + charsRead + " characters");
                System.out.println "[" + threadName + "] Execution time: " +
                    (executionTime / 1000.0) + " seconds");
            } else {
                System.out.println "[" + threadName + "] Failed to fetch content");
            }
        }
    }

    public int getLinesRead() {
        return linesRead;
    }

    public int getCharsRead() {
        return charsRead;
    }

    public long getExecutionTime() {
        return executionTime;
    }

    public boolean isSuccess() {
        return success;
    }
}

```

3.3 Файл WebContentFetcher.java

// Task 2: Многопоточное приложение для загрузки содержимого веб-сайтов

```

public class WebContentFetcher {

    public static void main(String[] args) {
        System.out.println("\n");
        System.out.println("Task 2: Multi-threaded Web Content Fetcher");
        System.out.println("Concurrent website content downloading");
        System.out.println("\n");

        // Создаем читателей для разных сайтов
        WebsiteReader reader1 = new WebsiteReader("http://www.example.com", "Thread-1");
        WebsiteReader reader2 = new WebsiteReader("http://www.example.org", "Thread-2");

        // Создаем потоки
        Thread thread1 = new Thread(reader1);
        Thread thread2 = new Thread(reader2);

        System.out.println("Starting concurrent website fetching...\n");

        long startTime = System.currentTimeMillis();

        // Запускаем потоки одновременно
        thread1.start();
        thread2.start();

        // Ожидаем завершения обоих потоков
        try {
            thread1.join();
            thread2.join();
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted: " + e.getMessage());
        }

        long totalTime = System.currentTimeMillis() - startTime;

        // Выводим итоговую статистику
        System.out.println("\n" + "=".repeat(60));
        System.out.println("SUMMARY:");
        System.out.println("=".repeat(60));
        System.out.println("Thread-1 status: " + (reader1.isSuccess() ? "SUCCESS" : "FAILED"));
        System.out.println("Thread-2 status: " + (reader2.isSuccess() ? "SUCCESS" : "FAILED"));
        System.out.println("\nTotal execution time: " + (totalTime / 1000.0) + " seconds");

        if (reader1.isSuccess() && reader2.isSuccess()) {
            int totalLines = reader1.getLinesRead() + reader2.getLinesRead();
            int totalChars = reader1.getCharsRead() + reader2.getCharsRead();
        }
    }
}

```

```

        System.out.println("Total lines read: " + totalLines);
        System.out.println("Total characters read: " + totalChars);
    }

    System.out.println("=".repeat(60) + "\n");

    // Дополнительная демонстрация с большим количеством потоков
    demonstrateMultipleThreads();
}

private static void demonstrateMultipleThreads() {
    System.out.println("\n=====");
    System.out.println("Extended Demo: Multiple Concurrent Threads");
    System.out.println("=====");

    String[] urls = {
        "http://www.example.com",
        "http://www.example.org",
        "http://www.example.net"
    };

    Thread[] threads = new Thread[urls.length];
    WebsiteReader[] readers = new WebsiteReader[urls.length];

    // Создаем и запускаем потоки
    for (int i = 0; i < urls.length; i++) {
        readers[i] = new WebsiteReader(urls[i], "Thread-" + (i + 1));
        threads[i] = new Thread(readers[i]);
        threads[i].start();
    }

    // Ожидаем завершения всех потоков
    for (int i = 0; i < threads.length; i++) {
        try {
            threads[i].join();
        } catch (InterruptedException e) {
            System.err.println("Thread interrupted: " + e.getMessage());
        }
    }

    System.out.println("\n" + "=".repeat(60));
    System.out.println("All threads completed!");
    System.out.println("=".repeat(60) + "\n");
}
}

```

3.4 Файл App.java

```

// Главный класс для демонстрации обеих задач ЛР4
public class App {

    public static void main(String[] args) {
        System.out.println("\n=====");
        System.out.println("Лабораторная работа 4");
        System.out.println("Пакет java.net. Сетевые приложения Java");
        System.out.println("=====");

        // Задание 1: URL Parser
        System.out.println("=====");
        System.out.println("ЗАДАНИЕ 1");
        System.out.println("=====");

        URLParser.main(args);

        // Пауза между заданиями
        pause(2000);

        // Задание 2: Multi-threaded Web Content Fetcher
        System.out.println("\n=====");
        System.out.println("ЗАДАНИЕ 2");
        System.out.println("=====");

        WebContentFetcher.main(args);

        System.out.println("\n=====");
        System.out.println("Все задания выполнены успешно!");
        System.out.println("=====");
    }

    private static void pause(int milliseconds) {
        try {

```

```
    Thread.sleep(milliseconds);  
} catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}}
```


4 Результат работы программы

Лабораторная работа 4
Пакет java.net. Сетевые приложения Java

ЗАДАНИЕ 1

Task 1: URL Component Parser Demo
Java Networking (java.net.URL)

=====
Parsing URL: http://www.example.com
=====

 URL Components:

Protocol : http
Host : www.example.com
Port : 80 (default for http)
Path : /
File : /
Authority : www.example.com

Complete URL : http://www.example.com

=====
Parsing URL: https://www.example.com:8080/api/users
=====

 URL Components:

Protocol : https
Host : www.example.com
Port : 8080 (explicit)
Default Port : 443
Path : /api/users
File : /api/users
Authority : www.example.com:8080

Complete URL : https://www.example.com:8080/api/users

```
=====
Parsing URL: http://www.example.com/search?q=java&lang=en
=====
```

 URL Components:

```
-----
Protocol           : http
Host               : www.example.com
Port              : 80 (default for http)
Path              : /search
File              : /search?q=java&lang=en
Query Parameters  : q=java&lang=en
Authority          : www.example.com
-----
Complete URL       : http://www.example.com/search?q=java&lang=en
```

```
=====
Parsing URL: ftp://ftp.example.com/pub/docs/manual.pdf
=====
```

 URL Components:

```
-----
Protocol           : ftp
Host               : ftp.example.com
Port              : 21 (default for ftp)
Path              : /pub/docs/manual.pdf
File              : /pub/docs/manual.pdf
Authority          : ftp.example.com
-----
Complete URL       : ftp://ftp.example.com/pub/docs/manual.pdf
```

```
=====
Parsing URL: http://localhost:3000/index.html
=====
```

 URL Components:

```
-----
Protocol           : http
Host               : localhost
Port              : 3000 (explicit)
Default Port       : 80
Path              : /index.html
File              : /index.html
Authority          : localhost:3000
-----
Complete URL       : http://localhost:3000/index.html
```

```

=====
Parsing                                                                    URL:
https://docs.oracle.com/javase/tutorial/networking/urls/index.html#DOWNLOADING
=====

📄 URL Components:
-----
Protocol           : https
Host               : docs.oracle.com
Port               : 443 (default for https)
Path               : /javase/tutorial/networking/urls/index.html
File               : /javase/tutorial/networking/urls/index.html
Fragment/Anchor    : DOWNLOADING
Authority          : docs.oracle.com
-----

Complete          URL                                                    :
https://docs.oracle.com/javase/tutorial/networking/urls/index.html#DOWNLOADING

=====
Parsing                                                                    URL:
https://user:pass@www.example.com:8443/path/to/resource.html?id=123&type=test#section2
=====

📄 URL Components:
-----
Protocol           : https
Host               : www.example.com
Port               : 8443 (explicit)
Default Port       : 443
Path               : /path/to/resource.html
File               : /path/to/resource.html?id=123&type=test
Query Parameters   : id=123&type=test
Fragment/Anchor    : section2
Authority          : user:pass@www.example.com:8443
-----

Complete          URL                                                    :
https://user:pass@www.example.com:8443/path/to/resource.html?id=123&type=test#section2

=====
Parsing URL: http://invalid url with spaces
=====
ERROR: Invalid URL format!
Reason: unknown protocol: http

=====
URL parsing demonstration completed!

```

=====

ЗАДАНИЕ 2

```
Task 2: Multi-threaded Web Content Fetcher
Concurrent website content downloading
```

Starting concurrent website fetching...

```
[Thread-1] Starting: http://www.example.com
[Thread-2] Starting: http://www.example.org
[Thread-2] Connected successfully
[Thread-2] Completed: 1 lines, 512 characters
[Thread-2] Execution time: 0.192 seconds
[Thread-1] Connected successfully
[Thread-1] Completed: 1 lines, 512 characters
[Thread-1] Execution time: 0.283 seconds
```

=====

SUMMARY:

=====

Thread-1 status: SUCCESS
Thread-2 status: SUCCESS

Total execution time: 0.283 seconds
Total lines read: 2
Total characters read: 1024

=====

```
Extended Demo: Multiple Concurrent Threads
```

```
[Thread-1] Starting: http://www.example.com
[Thread-2] Starting: http://www.example.org
[Thread-3] Starting: http://www.example.net
[Thread-2] Connected successfully
[Thread-2] Completed: 1 lines, 512 characters
[Thread-2] Execution time: 0.076 seconds
[Thread-3] Connected successfully
```

```
[Thread-3] Completed: 1 lines, 512 characters
[Thread-3] Execution time: 0.099 seconds
[Thread-1] Connected successfully
[Thread-1] Completed: 1 lines, 512 characters
[Thread-1] Execution time: 0.151 seconds
```

```
=====
All threads completed!
=====
```

Все задания выполнены успешно!

Выводы

В ходе выполнения лабораторной работы были изучены основы сетевого программирования в Java с использованием пакета `java.net`. Были освоены методы парсинга и анализа структуры URL с помощью класса `URL`, а также практические аспекты подключения и чтения данных из сетевых ресурсов.

Особое внимание было уделено разработке многопоточного приложения для параллельной загрузки веб-контента. В процессе работы закрепились навыки создания и управления потоками (`Thread`, `Runnable`), их синхронизации для корректного вывода данных и обработки различных сетевых исключений (`MalformedURLException`, `IOException`).

Практическая реализация URL-парсера и многопоточного загрузчика наглядно продемонстрировала применение изученных классов и принципов для решения реальных задач сетевого взаимодействия.